ECA1014 – MICROPROCESSOR AND MICROCONTROLLER

CO1 Constraint-Based Problem-Solving Assessment

NAME: Murali Krishnan N

Reg No:192524090

Assignment – 1:

Constraint-Based Problem Solving with 8-bit Processor Assembly Language Programming

Course Outcome – CO1

Understand microprocessor architecture and identify different instruction set classifications and

addressing modes for arithmetic, logic, data transfer, and branch operations.

Nature of Assessment:

Open-Ended Engineering Problem-Solving / Assembly Programming Assignment

Duration: 1 Week

Maximum Marks: 100

Primary Bloom's Levels: K3 – Apply | K4 – Analyze | K5 – Evaluate | K6 – Create/Design

Permitted: Writing the 8-bit Assembly Language Program (ALP) is permitted. All calculations,

instruction selection, addressing-mode analysis and expected results shall be worked out

manually.

Contents

Section	Title	Marks
0	Design assumptions and specification notes	–
1	Stage 1 – Interpret the system requirement (block diagram, requirement statement)	15
2	Stage 2 – Classify and select instructions	15
3	Stage 3 – Design the operational logic (flowchart, Design Decision 1)	15
4	Stage 4 – Addressing-mode selection under constraints (Design Decision 2)	15
5	Stage 5 – Develop the 8-bit assembly language program	25
6	Stage 6 – Test, verify and critically evaluate	15
7	Final engineering decision	–

0. Design Assumptions and Specification Notes

The assignment fixes the memory map for the inputs, the threshold, the mask, the status byte and the arithmetic result. It leaves several details open. The decisions below were made once, at the start, and are used consistently throughout the report.

No.	Item	Decision and reason
A1	Processor	Intel 8085. Only instructions that the 8080 also has are used, so the program is valid on any Intel 8-bit basic processor. A 3 MHz clock is assumed for timing (T-state = 0.333 µs).
A2	Sensor inputs	2050H holds the temperature (Input 1) and 2051H holds the humidity (Input 2), matching the Temperature and Humidity columns of the test-case table. Both are unsigned 8-bit values.
A3	Threshold and limits	The single threshold T at 2052H is the WARNING limit. The CRITICAL limit is 2T. NORMAL: Sum < T. WARNING: T ≤ Sum < 2T. CRITICAL: Sum ≥ 2T. With T = 10H the limits are 10H and 20H, which gives the NORMAL, WARNING and CRITICAL results expected for test cases 1, 2 and 3. No second limit byte is needed.
A4	Overflow	If the 8-bit addition produces a carry (Sum greater than FFH), the true sum is certainly above any limit, so the state is CRITICAL. The stored 8-bit result at 3051H is the truncated sum.
A5	Control bits	Bit 0 = FAN, bit 1 = ALARM, bit 2 = HUMIDIFIER. NORMAL: all off (00H). WARNING: FAN and HUMIDIFIER on (05H). CRITICAL: FAN and ALARM on, HUMIDIFIER off (03H), because adding moisture to an overheating room makes the condition worse. The brief does not define these patterns, so this is a stated design choice.
A6	Location of control bits	The brief allocates 3051H to the arithmetic result, and the verification table also mentions 3051H for the control bits. One byte cannot hold both, so the arithmetic result stays at 3051H and the control byte is stored at 3052H.
A7	Masked result	The mask at 2053H is ANDed with the 8-bit sum and the result is stored at 3060H (the brief's "mask update 3060H onward"). The test mask value is 0FH.
A8	Program origin	The program starts at 2000H. Locations 2050H to 2053H are loaded before execution, by the sensor interface for 2050H and 2051H and by the engineer for 2052H and 2053H.
A9	Optional items	The event counter (INR M) and the display-buffer copy loop (DCR C, JNZ) listed in Stage 2 are an optional extension. They are given in Section 5.7 and are not part of the verified 58-byte core program.

Stage 1 – Interpret the System Requirement

1.1 Role of the 8-bit processor and the major functional blocks

The 8085 is the decision-making element of the Monitoring and Control Unit. It does not measure anything itself. It reads two digitised sensor values from memory, adds them, classifies the result against a stored threshold, and writes a status code and a set of control bits for the actuators. Four functional blocks are involved.

Block	Function in this system
Sensor interface	Converts temperature and humidity to 8-bit values and places them at 2050H and 2051H.
Memory (program and data)	Holds the program from 2000H, the inputs at 2050H–2051H, the threshold at 2052H, the mask at 2053H, and the results at 3050H, 3051H, 3052H and 3060H.
8085 CPU	Fetches and decodes the program, performs the addition, comparison and masking in the ALU, and takes the three-way decision.
Output interface and actuators	The status byte (3050H) and control byte (3052H) drive the FAN, ALARM and HUMIDIFIER.

1.2 How the ALU, control unit, registers and buses take part

Element	Participation in processing the sensor values
ALU	ADD M adds temperature and humidity. SUB C and CMP C subtract to classify the sum. ANA B applies the mask. Every operation sets or clears the flags.
Accumulator and temporary register	A is the implied source and destination of the arithmetic and logic instructions. The temporary register holds the second operand inside the ALU and is not visible to the programmer.
Flag register	S, Z, AC, P and CY are updated by the ALU. Only CY is tested by the program, by JC, to select NORMAL, WARNING or CRITICAL.
Instruction register and decoder	Hold the opcode read from memory and decode it into the micro-operations of the instruction.
Timing and control unit	Sequences the machine cycles and generates ALE, RD', WR' and IO/M'. It also tests the CY flag during JC to decide whether the program counter is loaded with the jump address.
Program counter	Holds the address of the next instruction. It is incremented after each byte fetch and is overwritten by JC and JMP.
Registers B, C, D, E, H, L	B holds the sum, C the threshold, D the status code, E the control byte, and HL is the memory pointer.
Address bus (16 bit)	Carries 2000H onward for instruction fetches, 2050H–2053H for the inputs and 3050H–3060H for the outputs.
Data bus (8 bit)	Carries opcodes, operand bytes, the sensor values and the results between memory and the CPU. Its 8-bit width matches the 8-bit sensor data.
Control bus	RD' and WR' mark memory reads and writes. IO/M' is low because all accesses are memory accesses. ALE latches the low address byte from the multiplexed AD7–AD0 lines.

1.3 Why an 8-bit processor suits this data

- Each sensor reading is one byte (0 to 255), so one memory read, one register and one ALU operation handle a complete value. No multi-byte arithmetic is needed.
- The 8-bit data bus transfers a full reading in a single read cycle. The 16-bit address bus (64 KB) is far more than the few dozen bytes used here.
- ADD, SUB, CMP and ANA operate directly on 8-bit data, and the carry flag signals the one case that 8 bits cannot hold (sum above FFH).
- The task needs one addition, two comparisons and one logical AND. This is far below the capability of a wider processor, so an 8-bit device meets the need at low cost, low power and low component count.

1.4 Input, processing, decision and output flow

Phase	Operation	Locations
Input	Read temperature, humidity, threshold and mask.	2050H, 2051H, 2052H, 2053H
Processing	Sum = Temperature + Humidity (8-bit, carry captured). Masked = Sum AND Mask.	3051H, 3060H
Decision	Compare Sum with T and 2T using the carry flag. Select NORMAL, WARNING or CRITICAL.	Flags CY
Output	Write status code and control bits.	3050H, 3052H

1.5 Constraints and their influence

Constraint	Influence on the design
Instruction classes: data transfer, arithmetic, logic and branch must all appear	The program uses MOV, MVI, LXI, LDA, STA (data transfer), ADD, SUB, CMP, INX (arithmetic), ANA (logic), JC, JMP (branch) and HLT (control).
Registers limited to A, B, C, D, E, H, L	No stack and no register pairs beyond HL are needed. Each register gets one fixed role (Section 5.2). The stack pointer is never used.
Program size 256 bytes or fewer	A shared output block with JMP instead of three copies keeps the program at 58 bytes (22.7 percent of the budget).
Memory map is fixed	Inputs at 2050H–2053H are adjacent, so one pointer (HL) can walk them. Outputs at 3050H and 3051H are written with direct addressing.
Five addressing modes must be shown	Immediate (LXI, MVI), register (MOV, SUB), direct (LDA, STA), register indirect (MOV A,M, ADD M) and implied (HLT, INX H).
No software tools	The program is short enough to hand-assemble and trace. Every opcode and flag is derived by hand.
Threshold read from memory	CPI with an immediate value is not usable for the limit. The threshold must be loaded from 2052H.

1.6 Evolution of microprocessors

Stage	Year and features	Engineering improvement introduced
Intel 4004	1971. 4-bit, about 2,300 transistors, clock about 740 kHz. First single-chip microprocessor.	Put a complete CPU on one chip, which made programmable control practical in small products such as calculators.
Intel 8080	1974. 8-bit, about 6,000 transistors, 16-bit address bus (64 KB). Needed +5 V, −5 V and +12 V supplies and external clock and bus-controller chips.	An 8-bit data path and 64 KB address space. This is the instruction set used in this report.
Intel 8085	1976. 8-bit, single +5 V supply, on-chip clock generator and system controller, serial I/O lines and five interrupt inputs.	Fewer support chips and a simpler board. This is the processor used here.
Intel 8086	1978. 16-bit, about 29,000 transistors, 20-bit address bus (1 MB).	Wider data path, larger memory through segmentation, and an instruction queue so fetching overlaps execution.
Intel 80386	1985. 32-bit, about 275,000 transistors, 32-bit address bus (4 GB).	Protected mode, paging and virtual memory, which allow multitasking operating systems.

Two representative improvements are the 8080 to 8085 step (integration of clock and bus control, single supply, which reduces chip count and cost) and the 8086 to 80386 step (protected mode and paging, which protect programs from each other and extend memory). Modern processors add pipelines, caches and multiple cores, but the register, ALU, flag and instruction-cycle ideas used in this report remain the same.

1.7 Stage 1 deliverable: 8085 functional block diagram

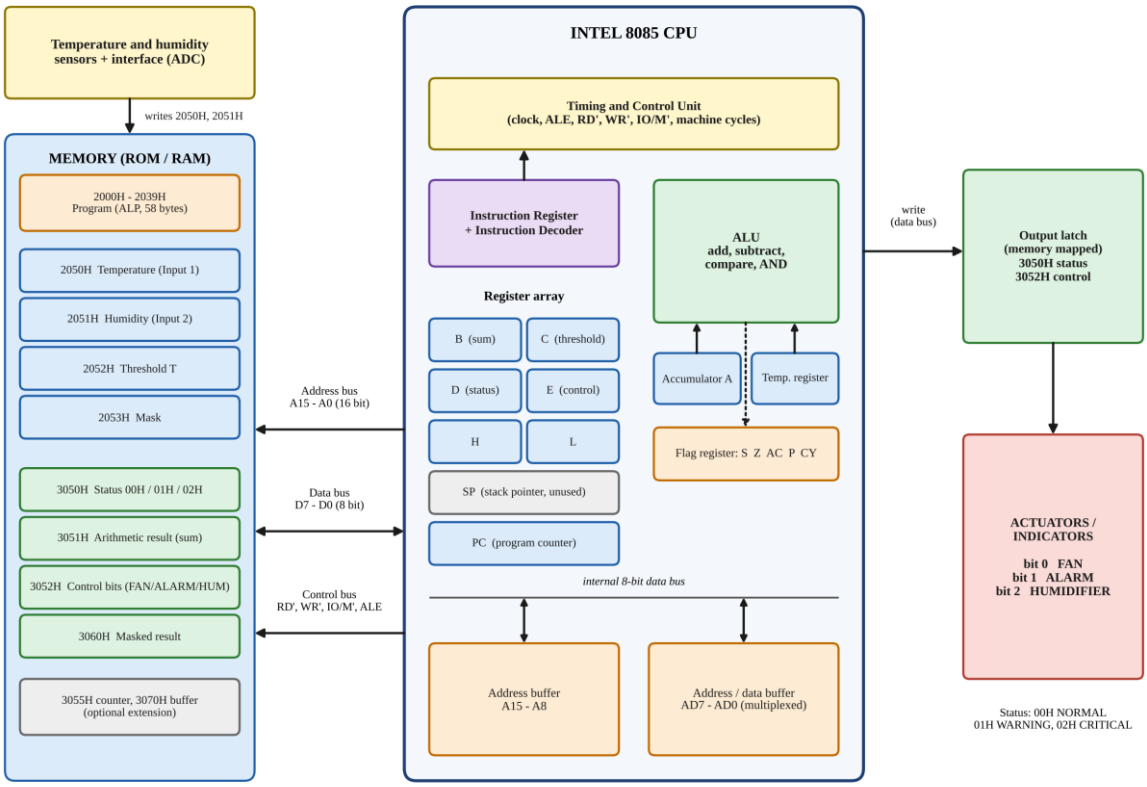


Figure 1. Functional block diagram of the 8085-based Monitoring and Control Unit, showing the CPU blocks, the three buses, memory and the output interface.

1.8 System Requirement Statement

System Requirement Statement. The system shall read the 8-bit temperature at 2050H and the 8-bit humidity at 2051H, form their 8-bit sum with carry-out treated as overflow, and store the sum at 3051H. It shall classify the sum against the threshold T held at 2052H as NORMAL (Sum < T), WARNING (T ≤ Sum < 2T) or CRITICAL (Sum ≥ 2T or carry-out), write the status code 00H, 01H or 02H to 3050H, and write the FAN, ALARM and HUMIDIFIER control byte to 3052H. It shall also AND the sum with the mask held at 2053H and store the result at 3060H. The program shall use only registers A, B, C, D, E, H and L, shall occupy not more than 256 bytes, and shall contain data-transfer, arithmetic, logic and branch instructions and the immediate, register, direct, register indirect and implied addressing modes.

Stage 2 – Classify and Select Instructions

2.1 Instruction-selection table

Required operation	Class	Selected instruction	Addressing mode	Flags and data handling	Reason
Read temperature and humidity	Data transfer	LXI H,2050H MOV A,M INX H ADD M	Immediate (pointer load) Register indirect	LXI, MOV, INX change no flags. ADD M sets all flags.	One pointer reaches 2050H–2052H. 6 bytes and 30 T-states, against 8 bytes and 34 T-states for LDA, MOV B,A, LDA, ADD B.
Compare with limits	Arithmetic / compare	SUB C CMP C	Register	Both set S, Z, AC, P, CY. CY = 1 when A < C (borrow). CMP leaves A unchanged.	CY gives a direct unsigned less-than test. SUB C then CMP C tests Sum ≥ 2T as (Sum − T) ≥ T, with no overflow.
Increment event counter (optional)	Arithmetic	INR M	Register indirect	Sets S, Z, AC, P. CY is not affected. 8-bit wrap FFH to 00H.	One-byte increment of a memory counter without using A and without disturbing CY.
Generate or clear control bits	Logic	ANA B (mask) ORI, ANI, XRI, CMA	Register (ANA B) Immediate (ORI, ANI)	ANA: CY = 0, AC = 1, S, Z, P from result.	ANA with a mask clears every bit where the mask is 0 and keeps the others. ORI 04H would set HUMIDIFIER and ANI FDH would clear ALARM. Here each state loads a complete byte with MVI, which is shorter.
Select NORMAL / WARNING / CRITICAL	Branch	JC, JMP	Direct (16-bit target in the instruction)	JC tests CY. No flag is changed.	One flag decides all three classes. JMP merges the paths into one output block.
Copy values to display buffer (optional)	Data transfer	MOV A,M STAX D	Register indirect	No flags change.	HL points at the source and DE at the buffer. Both pointers increment with INX.
Loop over display buffer (optional)	Branch	DCR C JNZ LOOP	Register, direct target	DCR sets Z, leaves CY. JNZ tests Z.	Counter 3 down to 0. Z = 1 ends the loop. The 8085 has no relative jump, so JNZ uses an absolute address.

2.2 Classification of every instruction in the program

Instruction	Opcode	Bytes	T-states	Class	Addressing mode	Flags affected
LXI H,addr	21	3	10	Data transfer	Immediate	None
MOV A,M / MOV C,M	7E / 4E	1	7	Data transfer	Register indirect	None
MOV B,A / MOV A,B	47 / 78	1	4	Data transfer	Register	None
MOV A,D / MOV A,E	7A / 7B	1	4	Data transfer	Register	None
MVI D,data / MVI E,data	16 / 1E	2	7	Data transfer	Immediate	None
LDA addr	3A	3	13	Data transfer	Direct	None
STA addr	32	3	13	Data transfer	Direct	None

Instruction	Opcode	Bytes	T-states	Class	Addressing mode	Flags affected
INX H	23	1	6	Arithmetic	Register pair (implied in opcode)	None
ADD M	86	1	7	Arithmetic	Register indirect	S, Z, AC, P, CY
SUB C	91	1	4	Arithmetic	Register	S, Z, AC, P, CY
CMP C	B9	1	4	Arithmetic (compare)	Register	S, Z, AC, P, CY
ANA B	A0	1	4	Logic	Register	S, Z, P; CY = 0, AC = 1
JC addr	DA	3	10 taken / 7 not taken	Branch	Direct	None
JMP addr	C3	3	10	Branch	Direct	None
HLT	76	1	5	Other control	Implied	None

2.3 Alternative instructions that were rejected

Rejected alternative	Reason for rejection under the constraints	What was used instead
CPI 10H (compare immediate)	Hard-codes the limit into the program. The brief requires the threshold to be read from 2052H so it can change without reprogramming.	MOV C,M then CMP C
LDA 2050H, MOV B,A, LDA 2051H, ADD B	Works, but costs 8 bytes and 34 T-states against 6 bytes and 30 T-states, and ties up register B early.	LXI H, MOV A,M, INX H, ADD M
ADD A (or RLC) to form 2T, then compare	Needs an extra register and overflows when T is above 7FH, which would give a false NORMAL or WARNING.	SUB C then CMP C, which tests Sum ≥ 2T without overflow
JZ or JNZ to classify	Z only tells equality. It cannot give a less-than decision, so extra compares would be needed.	JC on the carry flag
Three separate output sequences, one per state	Each copy is 16 bytes. Two extra copies cost 32 bytes more than the two 3-byte JMP instructions that replace them.	One shared STORE block

Stage 3 – Design the Operational Logic

3.1 Decision logic (flowchart)

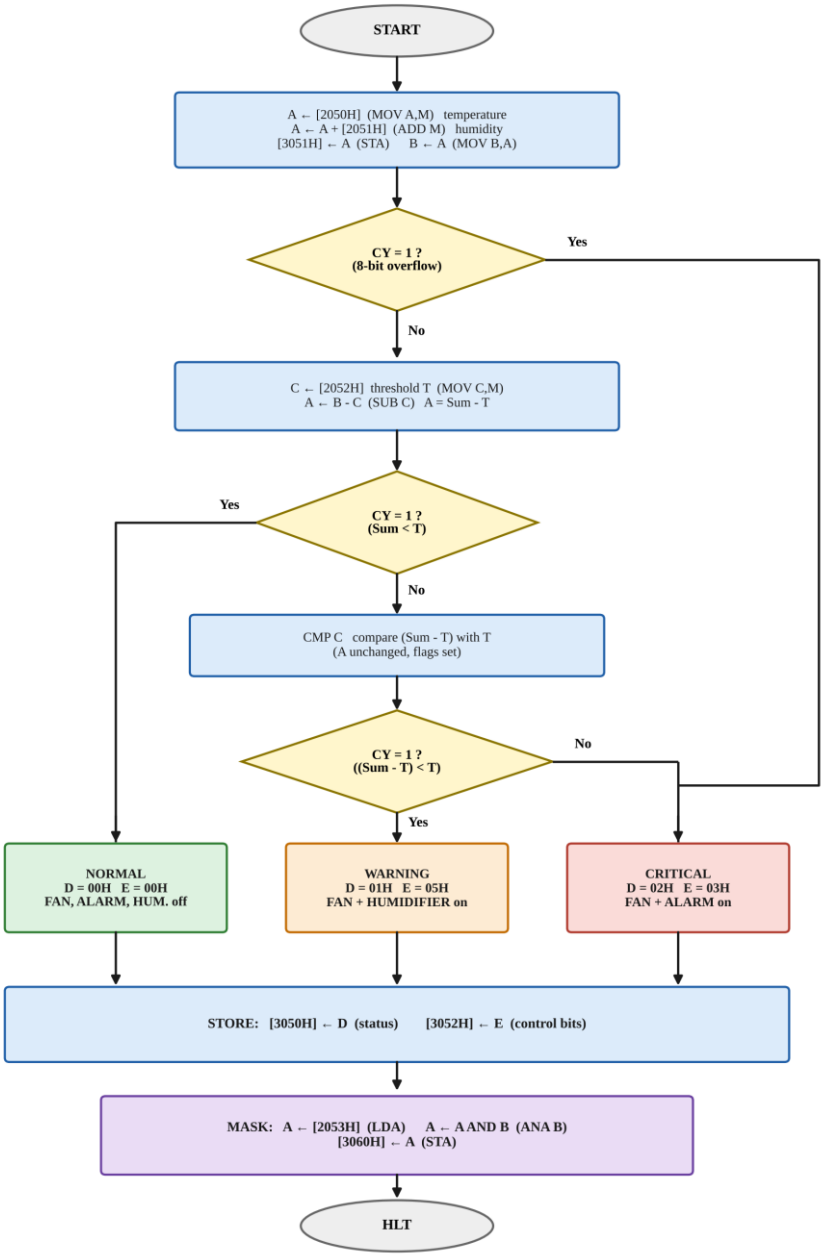


Figure 2. Flowchart of the NORMAL, WARNING and CRITICAL decision logic. D holds the status code and E holds the control byte.

3.2 Conditions for the status code and the FAN, ALARM and HUMIDIFIER bits

State	Condition (flags)	3050H status	FAN bit 0	ALARM bit 1	HUMID. bit 2	3052H control
NORMAL	Sum < T (CY = 1 after SUB C)	00H	Cleared	Cleared	Cleared	00H

State	Condition (flags)	3050H status	FAN bit 0	ALARM bit 1	HUMID. bit 2	3052H control
WARNING	T ≤ Sum < 2T (CY = 0 after SUB C, CY = 1 after CMP C)	01H	Set	Cleared	Set	05H
CRITICAL	Sum ≥ 2T (CY = 0 after both), or carry-out of the addition	02H	Set	Set	Cleared	03H

3.3 Flags used by the comparison and branch instructions

Flag and instruction	Meaning	Used by
CY after ADD M	1 when T + H exceeds FFH (carry out of bit 7).	JC CRIT
CY after SUB C	1 when Sum < T, because a borrow was needed.	JC NORM
CY after CMP C	1 when (Sum − T) < T. Equal values give CY = 0.	JC WARN
Z	Set when a result is zero. Sum = T gives Z = 1 after SUB C. Sum = 2T gives Z = 1 after CMP C. Z is not tested, because CY = 0 already places both cases in the upper class.	Not tested
S, P, AC	Updated by the ALU instructions but not needed for an unsigned less-than decision.	Not tested

3.4 How arithmetic, logic and branch instructions transform the input

Take temperature 08H and humidity 09H with T = 10H and mask 0FH. ADD M gives 11H and CY = 0. The program stores 11H at 3051H and keeps it in B. SUB C computes 11H − 10H = 01H with no borrow, so CY = 0 and the NORMAL branch is not taken. CMP C compares 01H with 10H, and since 01H is smaller, CY = 1 and JC WARN is taken. The data-transfer instructions MVI D,01H and MVI E,05H load the status and control bytes, and STA writes them to 3050H and 3052H. Finally LDA 2053H and ANA B give 0FH AND 11H = 01H, which STA writes to 3060H.

In short, arithmetic instructions produce the numbers and set CY, the branch instructions turn CY into a decision, the data-transfer instructions create and store the outputs, and the logic instruction produces the masked result.

3.5 Minimum number of processing steps and redundancy removed

The minimum is nine major steps: (1) fetch the temperature, (2) add the humidity, (3) store the arithmetic result, (4) test for overflow, (5) fetch the threshold, (6) classify with SUB C and CMP C, (7) load the status and control bytes, (8) write them to 3050H and 3052H, and (9) mask and store the result at 3060H. The following redundant operations were removed.

- **Doubling the threshold.** Forming 2T takes about three extra instructions (3 bytes, 12 T-states) and risks overflow. Testing (Sum − T) ≥ T removes both.
- **Reloading HL for 2052H.** INX H costs 1 byte and 6 T-states, against 3 bytes and 10 T-states for LXI H,2052H.

- **Reading the sum back from memory.** Keeping the sum in B makes MOV A,B (4 T-states) replace LDA 3051H (13 T-states).
- **Three copies of the output code.** A shared STORE block saves 32 − 6 = 26 bytes.

Design Decision 1 – Branch strategy

Recommended strategy: cascaded conditional branching on the carry flag only (JC), with the CRITICAL case placed as the fall-through path and one shared STORE block reached by JMP.

Criterion	Justification
Correctness	CY has an exact meaning for unsigned data: it is 1 when the first value is less than the second. The boundary cases come out correctly, because Sum = T gives CY = 0 after SUB C (WARNING) and Sum = 2T gives CY = 0 after CMP C (CRITICAL). The overflow case is handled by its own JC straight after the addition.
Readability	Each branch asks one question and the code runs top to bottom in the same order as Figure 2. Labels CRIT, WARN, NORM and STORE correspond to the boxes in the flowchart.
Instruction efficiency	Only one flag is used, so no extra instructions are needed to prepare other flags. NORMAL, the most common state, is placed next to STORE and needs no JMP (168 T-states, against 189 for WARNING and 186 for CRITICAL). The shared block saves 26 bytes.

Stage 4 – Addressing Mode Selection Under Constraints

4.1 Analysis of the five addressing modes

Mode	Required task	Instruction and operand form	Analysis and suitability
Immediate	Load threshold and mask values	In the program: LXI H,2050H; MVI D,02H; MVI E,03H. One-time set-up if memory is not preloaded: MVI A,10H; STA 2052H; MVI A,0FH; STA 2053H (10 bytes, not part of the logic).	The data is part of the instruction, so no extra memory access is needed. It suits constants known at assembly time (status codes, control bytes, pointers). It is not used to compare with T at run time, because T must stay changeable from 2052H.
Register	Intermediate calculations	MOV B,A; MOV A,B; SUB C; CMP C; ANA B; MOV A,D; MOV A,E	No memory cycle, so each instruction takes 4 T-states against 7 to 13 for memory modes. It keeps the sum in B and the threshold in C available without re-reading memory.
Direct	Access 2050H, 2051H, 2052H, 2053H, 3050H, 3051H	LDA 2053H; STA 3050H; STA 3051H; STA 3052H; STA 3060H. (LDA 2050H, 2051H and 2052H are the direct equivalents for the inputs.)	3 bytes and 13 T-states. The address is visible in the listing, which documents the output map. It is best for isolated locations such as 2053H and the outputs. The adjacent inputs 2050H–2052H are better reached through HL.
Register indirect	Access memory through HL	MOV A,M (2050H); ADD M (2051H); MOV C,M (2052H); STAX D in the optional loop	1-byte instructions of 7 T-states. INX H (6 T-states) steps the pointer, so three adjacent locations are reached for the cost of one LXI H.
Implied	Accumulator and flag related operation	HLT; INX H (register pair HL is part of the opcode). Others available: CMA, RAL, RAR, STC, CMC, DAA	No operand byte. CMA, STC and CMC work only on A or CY, so they are the cheapest way to complement the accumulator or set or toggle the carry. HLT stops instruction fetch at the end of the program.

Design Decision 2 – Addressing mode for each major data access

Data-access operation	Selected mode	Instruction	Reason
Read temperature (2050H)	Register indirect	MOV A,M	HL already points at 2050H. 1 byte, 7 T-states.
Read humidity (2051H) and add	Register indirect	INX H; ADD M	Reads and adds in one instruction with no extra register.
Read threshold (2052H)	Register indirect	INX H; MOV C,M	Continues the same pointer walk.
Read mask (2053H)	Direct	LDA 2053H	An isolated location read once. Direct addressing leaves HL free.
Intermediate values	Register	MOV, SUB, CMP, ANA	Fastest. No memory access.
Status and control constants	Immediate	MVI D, MVI E	The values are fixed for each state.

Data-access operation	Selected mode	Instruction	Reason
Write outputs (3050H, 3051H, 3052H, 3060H)	Direct	STA addr	Each output address appears in the listing. The brief asks for direct access to 3050H and 3051H.
Stop	Implied	HLT	No operand needed.

Cases where a different mode works but is less suitable

1. **Direct addressing for the sensor inputs** (LDA 2050H, MOV B,A, LDA 2051H, ADD B). It works but costs 8 bytes and 34 T-states, against 6 bytes and 30 T-states for register indirect.
2. **Immediate addressing for the threshold** (CPI 10H). It works for one fixed limit, but the limit would then be buried in the program and could not be changed through 2052H.
3. **Direct addressing to re-read the sum** (LDA 3051H instead of MOV A,B). It works but takes 13 T-states against 4, and uses 3 bytes against 1.
4. **Register indirect for the output stores** (LXI H,3050H; MOV M,D; INX H; MOV M,B; INX H; MOV M,E). This was evaluated honestly and is slightly better: 8 bytes against 11 and 43 T-states against 47. It was not adopted because the brief asks for direct access to 3050H and 3051H, and the 3-byte saving is small against the 198 bytes of spare memory. It is kept as an optional optimisation.

Stage 5 – Develop the 8-bit Assembly Language Program

5.1 Algorithm (pseudocode)

1. Set HL to 2050H. Load A with the temperature from [HL].
2. Increment HL to 2051H. Add [HL] (humidity) to A. CY records an overflow.
3. Store A at 3051H. Copy A to B.
4. If CY = 1, go to step 10 (CRITICAL).
5. Increment HL to 2052H. Load the threshold into C.
6. Copy B to A. Subtract C. A = Sum − T.
7. If CY = 1 (Sum < T), go to step 12 (NORMAL).
8. Compare A with C. If CY = 1 ((Sum − T) < T), go to step 11 (WARNING).
9. CRITICAL: D = 02H, E = 03H. Go to step 13.
10. (Entry point for overflow, same as step 9.)
11. WARNING: D = 01H, E = 05H. Go to step 13.
12. NORMAL: D = 00H, E = 00H.
13. Store D at 3050H and E at 3052H.
14. Load the mask from 2053H. AND with B. Store the result at 3060H.
15. Halt.

Step 10 is the same code as step 9. In the program the overflow branch JC CRIT jumps directly to the CRITICAL instructions at 2018H.

5.2 Register-allocation table

Register	Role	Justification
A (accumulator)	Working register: temperature, sum, Sum − T, status and control bytes in transit, mask and masked result.	ADD, SUB, ANA and the memory-access instructions LDA and STA all work through A.
B	Holds the 8-bit sum for the whole program.	The sum is needed three times (SUB, mask and store). Keeping it in B avoids re-reading memory.
C	Holds the threshold T.	SUB C and CMP C use T twice. One load serves both.
D	Status code (00H, 01H, 02H).	Holds the state until the shared STORE block, because the three branches meet at one block.
E	Control byte (00H, 05H, 03H).	Same reason as D.
H, L	Memory pointer: 2050H, then 2051H, then 2052H.	One pointer reaches three adjacent locations by INX H.
Flags (PSW)	CY carries every decision.	Preserved between the arithmetic instruction and its JC, because only data-transfer and INX instructions lie between them.
PC, SP	PC sequences the program. SP is not used.	No CALL, RET, PUSH or POP, so no stack is needed.

All seven programmer-visible 8-bit registers (A, B, C, D, E, H, L) are used and none has to be saved to memory.

5.3 Complete assembly program (hand-assembled)

Origin 2000H. Opcodes are written low byte first for 16-bit addresses, as the 8085 stores them. STA, LDA, MOV, MVI, INX and JC and JMP do not change the flags, which is why the carry from the addition can be tested after the store and register copies.

Address	Machine code	Label	Instruction	Comment
2000	21 50 20	START	LXI H,2050H	HL ← 2050H, pointer to temperature (immediate)
2003	7E		MOV A,M	A ← [2050H], temperature (register indirect)
2004	23		INX H	HL ← 2051H, point to humidity
2005	86		ADD M	A ← A + [2051H]; flags set, CY = carry out
2006	32 51 30		STA 3051H	[3051H] ← A, 8-bit arithmetic result (direct)
2009	47		MOV B,A	B ← sum, kept for classification and masking
200A	DA 18 20		JC CRIT	CY = 1: overflow, go to CRITICAL
200D	23		INX H	HL ← 2052H
200E	4E		MOV C,M	C ← [2052H], threshold T
200F	78		MOV A,B	A ← sum
2010	91		SUB C	A ← Sum − T; CY = 1 if Sum < T
2011	DA 26 20		JC NORM	Sum < T: NORMAL
2014	B9		CMP C	Compare (Sum − T) with T; CY = 1 if smaller
2015	DA 1F 20		JC WARN	T ≤ Sum < 2T: WARNING
2018	16 02	CRIT	MVI D,02H	D ← 02H, status CRITICAL
201A	1E 03		MVI E,03H	E ← 03H, FAN and ALARM on
201C	C3 2A 20		JMP STORE	Go to the shared output block
201F	16 01	WARN	MVI D,01H	D ← 01H, status WARNING
2021	1E 05		MVI E,05H	E ← 05H, FAN and HUMIDIFIER on
2023	C3 2A 20		JMP STORE	Go to the shared output block
2026	16 00	NORM	MVI D,00H	D ← 00H, status NORMAL
2028	1E 00		MVI E,00H	E ← 00H, all outputs off; falls into STORE
202A	7A	STORE	MOV A,D	A ← status
202B	32 50 30		STA 3050H	[3050H] ← status (direct)
202E	7B		MOV A,E	A ← control byte
202F	32 52 30		STA 3052H	[3052H] ← control byte
2032	3A 53 20		LDA 2053H	A ← mask (direct)
2035	A0		ANA B	A ← mask AND sum; CY = 0

Address	Machine code	Label	Instruction	Comment
2036	32 60 30		STA 3060H	[3060H] ← masked result
2039	76		HLT	Stop (implied)

5.4 Program size and timing

Item	Value	Working
Program size	58 bytes (3AH)	From 2000H to 2039H inclusive: 2039H − 2000H + 1 = 3AH = 58. 30 instructions.
Share of the 256-byte budget	22.7 percent	58 ÷ 256. Spare memory: 198 bytes.
Execution time, NORMAL path	168 T-states = 56.0 µs	Table 5.4a. At 3 MHz one T-state is 0.333 µs.
Execution time, WARNING path	189 T-states = 63.0 µs	Table 5.4a.
Execution time, CRITICAL path	186 T-states = 62.0 µs	Table 5.4a.
Execution time, overflow path	150 T-states = 50.0 µs	Skips the classification instructions.

Table 5.4a – T-state totals. The common start (LXI to MOV B,A) is 47 T-states: 10 + 7 + 6 + 7 + 13 + 4. The common finish (the STORE block) is 69 T-states: 4 + 13 + 4 + 13 + 13 + 4 + 13 + 5.

Path	Calculation	Total
NORMAL	47 + JC not taken 7 + INX 6 + MOV C,M 7 + MOV A,B 4 + SUB C 4 + JC taken 10 + MVI 7 + MVI 7 + STORE 69	168
WARNING	47 + 7 + 6 + 7 + 4 + 4 + JC not taken 7 + CMP 4 + JC taken 10 + MVI 7 + MVI 7 + JMP 10 + STORE 69	189
CRITICAL	47 + 7 + 6 + 7 + 4 + 4 + 7 + 4 + JC not taken 7 + MVI 7 + MVI 7 + JMP 10 + STORE 69	186
Overflow	47 + JC taken 10 + MVI 7 + MVI 7 + JMP 10 + STORE 69	150

5.5 Fetch–decode–execute analysis of representative instructions

The program counter starts at 2000H. In the 8085 an opcode fetch (OF) takes 4 T-states (6 for INX), and each further memory read (MR) or memory write (MW) takes 3. During every cycle the CPU places the address on A15–A8 and AD7–AD0, ALE latches the low byte, and RD' or WR' is pulsed.

Instruction (address)	Fetch	Decode	Execute	Cycles
LXI H,2050H (2000H)	PC = 2000H on the address bus. RD' low. Opcode 21H read into the instruction register. PC becomes 2001H.	The decoder identifies a 3-byte immediate load of HL and schedules two memory-read cycles.	MR 1: address 2001H, byte 50H loaded into L, PC = 2002H. MR 2: address 2002H, byte 20H loaded into H, PC = 2003H.	OF + MR + MR = 4 + 3 + 3 = 10 T

Instruction (address)	Fetch	Decode	Execute	Cycles
MOV A,M (2003H)	PC = 2003H. Opcode 7EH read. PC becomes 2004H.	Move from the memory location addressed by HL to A.	MR: HL (2050H) placed on the address bus, data 05H (case 1) read into A. No flags change.	OF + MR = 7 T
ADD M (2005H)	PC = 2005H. Opcode 86H read. PC becomes 2006H.	Add the memory operand addressed by HL to A.	MR: address 2051H, operand 03H into the temporary register. The ALU computes 05H + 03H = 08H, which goes to A. Flags set: CY = 0, Z = 0, S = 0.	OF + MR = 7 T
STA 3051H (2006H)	PC = 2006H. Opcode 32H read. PC becomes 2007H.	Store A at a direct 16-bit address; needs two address bytes and one write.	MR 1: low byte 51H into Z. MR 2: high byte 30H into W. MW: address WZ = 3051H, data A = 08H written. PC = 2009H.	OF + MR + MR + MW = 13 T
JC NORM (2011H)	PC = 2011H. Opcode DAH read. PC becomes 2012H.	Conditional jump. The control unit will test CY.	MR 1: low byte 26H into Z. CY = 1 in case 1, so MR 2 reads high byte 20H into W and PC ← 2026H. If CY were 0, MR 2 is skipped and PC = 2014H.	Taken 10 T, not taken 7 T
ANA B (2035H)	PC = 2035H. Opcode A0H read. PC becomes 2036H.	Logical AND of A with register B.	The ALU computes 0FH AND 08H = 08H into A (case 1). CY = 0, AC = 1, Z = 0, S = 0.	OF = 4 T

5.6 Major instructions and why they were selected

Instruction	Role in the program	Why selected
ADD M	Adds humidity to temperature straight from memory.	No second register or load is needed. It also sets CY for the overflow test.
SUB C and CMP C	Classify the sum against T and 2T.	Two instructions give both limits without doubling T and without overflow.
JC	Branches on the carry flag.	One flag, three decisions. It leaves all registers and flags unchanged.
MOV A,M and INX H	Walk through 2050H, 2051H and 2052H.	Cheapest way to read three adjacent bytes.
ANA B	Applies the mask.	Required logic operation. It clears all bits where the mask is 0.
STA and LDA	Write the four outputs and read the mask.	Direct addressing shows the output map clearly, as the brief requires.
MVI D and MVI E	Create the status and control bytes.	Constants known in advance are cheapest as immediate data.
JMP	Joins the WARNING and CRITICAL paths to STORE.	Saves 26 bytes against three copies of the output code.
HLT	Ends the program.	Stops fetching; implied addressing.

5.7 Optional extension: event counter and display buffer

This extension replaces the HLT at 2039H. It increments an event counter at 3055H (assumed to be 00H at power-up) and copies the three result bytes to a display buffer at 3070H–3072H, using a loop. It is not needed for the verified core, and it brings the program size to 78 bytes (4EH).

Address	Machine code	Label	Instruction	Comment
2039	21 55 30		LXI H,3055H	HL ← counter address
203C	34		INR M	[3055H] ← [3055H] + 1; CY unchanged
203D	21 50 30		LXI H,3050H	Source pointer: status, sum, control
2040	11 70 30		LXI D,3070H	Destination pointer: display buffer
2043	0E 03		MVI C,03H	Loop count
2045	7E	LOOP	MOV A,M	A ← source byte
2046	12		STAX D	[DE] ← A (register indirect)
2047	23		INX H	Next source byte
2048	13		INX D	Next destination byte
2049	0D		DCR C	Count down; Z = 1 when zero
204A	C2 45 20		JNZ LOOP	Repeat until count is zero
204D	76		HLT	Stop

Stage 6 – Test, Verify and Critically Evaluate

Each test case was traced by hand. Registers not yet loaded are shown as –. Flags are shown after each instruction; instructions that do not affect flags leave them unchanged. Test values: threshold 2052H = 10H and mask 2053H = 0FH.

6.1 Test case 1: temperature 05H, humidity 03H, threshold 10H (expected NORMAL)

2050H	2051H	2052H	2053H
05	03	10	0F

#	Instruction	A	B	C	D	E	HL	CY	Z	S	Memory effect
1	LXI H,2050H	–	–	–	–	–	2050	–	–	–	
2	MOV A,M	05	–	–	–	–	2050	–	–	–	A ← [2050H]
3	INX H	05	–	–	–	–	2051	–	–	–	
4	ADD M	08	–	–	–	–	2051	0	0	0	05 + 03 = 08
5	STA 3051H	08	–	–	–	–	2051	0	0	0	[3051H] = 08
6	MOV B,A	08	08	–	–	–	2051	0	0	0	
7	JC CRIT	08	08	–	–	–	2051	0	0	0	CY = 0, not taken
8	INX H	08	08	–	–	–	2052	0	0	0	
9	MOV C,M	08	08	10	–	–	2052	0	0	0	C = T = 10H
10	MOV A,B	08	08	10	–	–	2052	0	0	0	
11	SUB C	F8	08	10	–	–	2052	1	0	1	08 − 10 = F8, borrow
12	JC NORM	F8	08	10	–	–	2052	1	0	1	Taken, PC = 2026H
13	MVI D,00H	F8	08	10	00	–	2052	1	0	1	
14	MVI E,00H	F8	08	10	00	00	2052	1	0	1	
15	MOV A,D	00	08	10	00	00	2052	1	0	1	A = status
16	STA 3050H	00	08	10	00	00	2052	1	0	1	[3050H] = 00
17	MOV A,E	00	08	10	00	00	2052	1	0	1	
18	STA 3052H	00	08	10	00	00	2052	1	0	1	[3052H] = 00
19	LDA 2053H	0F	08	10	00	00	2052	1	0	1	A = mask
20	ANA B	08	08	10	00	00	2052	0	0	0	0F AND 08 = 08
21	STA 3060H	08	08	10	00	00	2052	0	0	0	[3060H] = 08
22	HLT	08	08	10	00	00	2052	0	0	0	Stop

3050H status	3051H sum	3052H control	3060H masked
00 (NORMAL)	08	00 (all off)	08

6.2 Test case 2: temperature 08H, humidity 09H, threshold 10H (expected WARNING)

2050H	2051H	2052H	2053H
08	09	10	0F

#	Instruction	A	B	C	D	E	HL	CY	Z	S	Memory effect
1	LXI H,2050H	–	–	–	–	–	2050	–	–	–	
2	MOV A,M	08	–	–	–	–	2050	–	–	–	A ← [2050H]
3	INX H	08	–	–	–	–	2051	–	–	–	
4	ADD M	11	–	–	–	–	2051	0	0	0	08 + 09 = 11
5	STA 3051H	11	–	–	–	–	2051	0	0	0	[3051H] = 11
6	MOV B,A	11	11	–	–	–	2051	0	0	0	
7	JC CRIT	11	11	–	–	–	2051	0	0	0	Not taken
8	INX H	11	11	–	–	–	2052	0	0	0	
9	MOV C,M	11	11	10	–	–	2052	0	0	0	C = 10H
10	MOV A,B	11	11	10	–	–	2052	0	0	0	
11	SUB C	01	11	10	–	–	2052	0	0	0	11 − 10 = 01
12	JC NORM	01	11	10	–	–	2052	0	0	0	Not taken
13	CMP C	01	11	10	–	–	2052	1	0	1	01 − 10, borrow; A unchanged
14	JC WARN	01	11	10	–	–	2052	1	0	1	Taken, PC = 201FH
15	MVI D,01H	01	11	10	01	–	2052	1	0	1	
16	MVI E,05H	01	11	10	01	05	2052	1	0	1	
17	JMP STORE	01	11	10	01	05	2052	1	0	1	PC = 202AH
18	MOV A,D	01	11	10	01	05	2052	1	0	1	
19	STA 3050H	01	11	10	01	05	2052	1	0	1	[3050H] = 01
20	MOV A,E	05	11	10	01	05	2052	1	0	1	
21	STA 3052H	05	11	10	01	05	2052	1	0	1	[3052H] = 05
22	LDA 2053H	0F	11	10	01	05	2052	1	0	1	A = mask
23	ANA B	01	11	10	01	05	2052	0	0	0	0F AND 11 = 01
24	STA 3060H	01	11	10	01	05	2052	0	0	0	[3060H] = 01
25	HLT	01	11	10	01	05	2052	0	0	0	Stop

3050H status	3051H sum	3052H control	3060H masked
01 (WARNING)	11	05 (FAN + HUMID.)	01

6.3 Test case 3: temperature 20H, humidity 15H, threshold 10H (expected CRITICAL)

2050H	2051H	2052H	2053H
20	15	10	0F

#	Instruction	A	B	C	D	E	HL	CY	Z	S	Memory effect
1	LXI H,2050H	–	–	–	–	–	2050	–	–	–	
2	MOV A,M	20	–	–	–	–	2050	–	–	–	A ← [2050H]
3	INX H	20	–	–	–	–	2051	–	–	–	
4	ADD M	35	–	–	–	–	2051	0	0	0	20 + 15 = 35
5	STA 3051H	35	–	–	–	–	2051	0	0	0	[3051H] = 35
6	MOV B,A	35	35	–	–	–	2051	0	0	0	
7	JC CRIT	35	35	–	–	–	2051	0	0	0	Not taken
8	INX H	35	35	–	–	–	2052	0	0	0	
9	MOV C,M	35	35	10	–	–	2052	0	0	0	C = 10H
10	MOV A,B	35	35	10	–	–	2052	0	0	0	
11	SUB C	25	35	10	–	–	2052	0	0	0	35 − 10 = 25
12	JC NORM	25	35	10	–	–	2052	0	0	0	Not taken
13	CMP C	25	35	10	–	–	2052	0	0	0	25 − 10 = 15, no borrow
14	JC WARN	25	35	10	–	–	2052	0	0	0	Not taken, PC = 2018H
15	MVI D,02H	25	35	10	02	–	2052	0	0	0	
16	MVI E,03H	25	35	10	02	03	2052	0	0	0	
17	JMP STORE	25	35	10	02	03	2052	0	0	0	PC = 202AH
18	MOV A,D	02	35	10	02	03	2052	0	0	0	
19	STA 3050H	02	35	10	02	03	2052	0	0	0	[3050H] = 02
20	MOV A,E	03	35	10	02	03	2052	0	0	0	
21	STA 3052H	03	35	10	02	03	2052	0	0	0	[3052H] = 03
22	LDA 2053H	0F	35	10	02	03	2052	0	0	0	A = mask
23	ANA B	05	35	10	02	03	2052	0	0	0	0F AND 35 = 05
24	STA 3060H	05	35	10	02	03	2052	0	0	0	[3060H] = 05
25	HLT	05	35	10	02	03	2052	0	0	0	Stop

3050H status	3051H sum	3052H control	3060H masked
02 (CRITICAL)	35	03 (FAN + ALARM)	05

6.4 Summary of the three test cases

Quantity	Case 1	Case 2	Case 3
Temperature, humidity (2050H, 2051H)	05H, 03H	08H, 09H	20H, 15H
Arithmetic result (3051H) and CY after ADD	08H, CY = 0	11H, CY = 0	35H, CY = 0
Sum − T and CY after SUB C	F8H, CY = 1	01H, CY = 0	25H, CY = 0
CY after CMP C	not executed	1 ((Sum − T) < T)	0 ((Sum − T) ≥ T)
Branch taken	JC NORM	JC WARN	none, falls to CRIT

Quantity	Case 1	Case 2	Case 3
Status byte (3050H)	00H NORMAL	01H WARNING	02H CRITICAL
Control byte (3052H)	00H	05H	03H
Logical result (3060H) = 0FH AND sum	08H	01H	05H
Expected condition met?	Yes	Yes	Yes
Instructions executed	22	25	25
T-states and time at 3 MHz	168, 56.0 µs	189, 63.0 µs	186, 62.0 µs

6.5 Boundary tests

The three required cases do not touch the limits. The following additional cases check the boundaries with T = 10H and mask 0FH.

Inputs	Sum (3051H), CY	SUB C result, CY	CMP C result	Status	Masked (3060H)
07H + 08H	0FH, 0	FFH, 1	not executed	00H NORMAL	0FH
08H + 08H (Sum = T)	10H, 0	00H, 0	00H vs 10H: CY = 1	01H WARNING	00H
0FH + 10H (Sum = 2T − 1)	1FH, 0	0FH, 0	0FH vs 10H: CY = 1	01H WARNING	0FH
10H + 10H (Sum = 2T)	20H, 0	10H, 0	10H vs 10H: CY = 0, Z = 1	02H CRITICAL	00H
C8H + 64H (overflow)	2CH, 1	not executed	not executed	02H CRITICAL	0CH

Every boundary case gives the intended class. In the overflow case JC CRIT is taken at 200AH after 57 T-states and the stored sum is the truncated 2CH.

6.6 Final verification table

Requirement	Specified	Achieved	Satisfied?	Evidence
Correct sensor data acquisition	2050H, 2051H	Read through HL with MOV A,M and ADD M	Yes	Trace steps 2 to 4 in all cases
Correct condition classification	Defined thresholds (T and 2T)	NORMAL, WARNING, CRITICAL for the three cases and five boundary cases	Yes	Sections 6.1 to 6.5
Correct control bits	Control bits (3051H in the brief's table)	00H, 05H, 03H stored at 3052H because 3051H holds the arithmetic result	Yes (see A6)	Trace steps 17 to 21
8-bit arithmetic result	Processed result at 3051H	08H, 11H, 35H; overflow handled by CY	Yes	Trace step 5, summary 6.4
Mask update	3060H onward	0FH AND sum stored at 3060H: 08H, 01H, 05H	Yes	Trace steps 19 to 21
Instruction-class coverage	Data transfer, arithmetic, logic, branch	MOV, MVI, LXI, LDA, STA; ADD, SUB, CMP, INX; ANA; JC, JMP; HLT	Yes	Table 2.2
Addressing-mode coverage	Immediate, register, direct, register indirect, implied	LXI and MVI; MOV B,A; LDA and STA; MOV A,M and ADD M; HLT and INX H	Yes	Table 4.1

Requirement	Specified	Achieved	Satisfied?	Evidence
Register constraint	A, B, C, D, E, H, L only	All seven used, no other register	Yes	Section 5.2
Program memory	256 bytes or fewer	58 bytes (22.7 percent)	Yes	Section 5.4
ALP correctness and conciseness	Correct 8-bit ALP	30 instructions, 168 to 189 T-states	Yes	Sections 5.3 and 6.4

7. Final Engineering Decision

Would you use these routines in the system?

YES.

The routines meet every requirement in the brief. The evidence from the complete design is as follows.

Consideration	Evidence
Correctness	All three required test cases and five boundary cases give the intended status, control byte, arithmetic result and masked result (Sections 6.1 to 6.5). Overflow is detected through CY, so a large sum cannot be mistaken for a small one.
Instruction efficiency	30 instructions. The worst path takes 189 T-states, which is 63 μs at 3 MHz. This is very short beside the response time of a temperature or humidity sensor, so the speed is more than enough.
Memory usage	58 bytes, 22.7 percent of the 256-byte budget, leaving 198 bytes. The optional extension raises this to 78 bytes, still within the limit.
Register usage	All seven registers used, each with one fixed role, and no spilling to the stack.
Addressing-mode suitability	Register indirect for the adjacent inputs, direct for isolated locations and outputs, immediate for constants, register for intermediate values, implied for HLT. Section 4.1 gives the figures that support each choice.
Verification evidence	Hand traces show register, flag and memory values after every instruction, and the verification table (Section 6.6) shows every requirement as satisfied.

Limitations noted, and the change that would address each

- The program runs once and halts. A real controller must read the sensors repeatedly. A loop back to START (or a timer interrupt such as RST 7.5) would fix this at a cost of 3 bytes for the JMP, and the HLT would be removed.
- The inputs are not range-checked. A faulty sensor reading is treated like a valid one. A validity test could be added if the sensor interface can report faults.
- The CRITICAL limit is fixed at 2T. If a separate limit is needed, a second byte at 2054H could hold it, and CMP M could replace CMP C.
- The output stores could use HL addressing and save 3 bytes and 4 T-states (Design Decision 2). This is minor and was not needed.

None of these limitations affects the correctness of the verified routines, so the answer is YES.